

freeQoS technical documentation

How freeQoS works, from the router to the screen: architecture, installation and sizing, network flows, measurements, NetFlow, shaping, API, security, operations and troubleshooting.

Contents

1. Overview
2. Architecture
3. Installation and updates
4. Routers and inventory
5. Measurements: throughput, latency, QoE
6. NetFlow traffic
7. Plans and shaping
8. Interface
9. API
10. Security
11. Operations
12. Troubleshooting
13. Settings reference

1. Overview

freeQoS is an **out-of-band** QoS/QoE controller for internet service providers running MikroTik routers. It reads the routers through their API, measures what each subscriber experiences, and installs CAKE queues to enforce the plans that were sold. It never sits in the traffic path.

It covers three jobs of an ISP:

- **See**: each subscriber's throughput, latency and bufferbloat, the load on every link and node, traffic by destination and by application (NetFlow).
- **Shape**: one CAKE queue per subscriber at the plan rate, under one parent queue per link, created and maintained automatically.
- **Integrate**: an operating API (`/api/v1`), a Preseem-compatible inventory API (`/model/v1`) and a consumption API (`/usage/v1`).

Design principles

Principle	What it means in practice
Out-of-band	No box in the path: freeQoS reads and writes the RouterOS configuration; a freeQoS outage cuts no subscriber.
Simulation by default	On first start nothing is written to the routers; writing is enabled in the interface, after reading the plan.
Strict ownership	freeQoS never modifies or deletes a RouterOS entry that does not carry <code>freeqos:managed</code> and the identifier of its instance.
Plan first	A subscriber is only throttled once a plan is pushed for it (API or Plans page); a subscriber that is only detected is observed.
Never make things up	A missing measurement is shown as missing ("-", "no ping reply"), never as a zero or a reassuring value.
Everything automatic	Adding a router is enough: CAKE queues, NetFlow export, discovery of subscribers and links happen by themselves.

Compared with Preseem and LibreQoS

	freeQoS	Preseem	LibreQoS
Place in the network	Out-of-band (RouterOS API)	Inline (bridge)	Inline (Linux server)
Where shaping happens	On the MikroTik routers (CAKE simple queues)	On the appliance	On the server (CAKE/HTB)
Latency measurement	Active probe (ping from the NAS, in the subscriber's VRF)	Passive (TCP)	Passive (TCP, eBPF)
Inventory API	Preseem-compatible <code>/model/v1</code>	Native	Files / integrations

A direct consequence of being out-of-band: latency is measured by a ping from the router, which requires the subscriber's box to answer pings (see section 5).

Key notions

The rest of this documentation relies on these words. Each one is covered in detail in its own section.

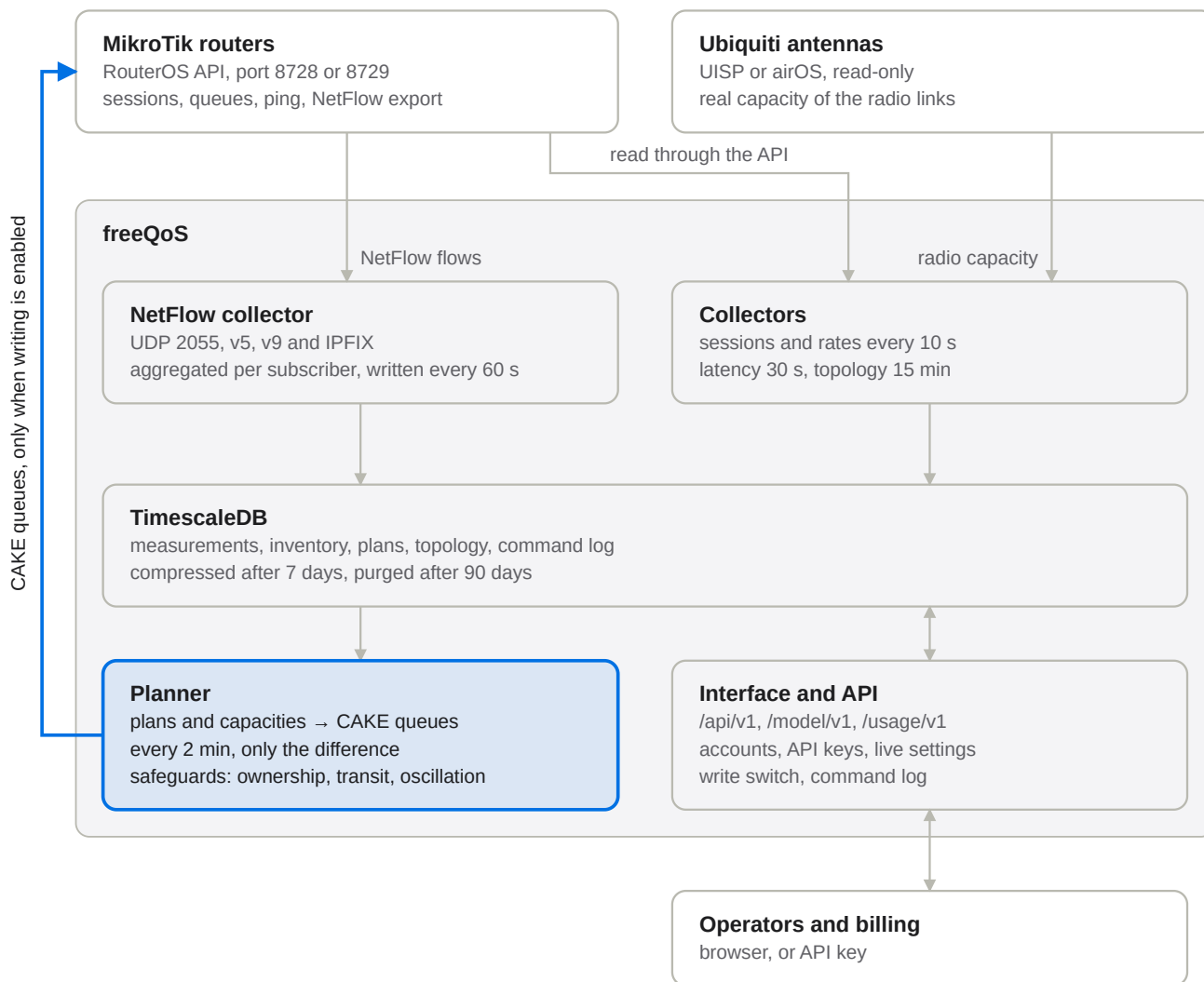
Notion	What it is	Why freeQoS cares
PoP (point of presence)	The router the subscribers of an area connect to	This is where freeQoS measures and limits each subscriber
Core, gateway	The routers between the PoPs and the internet	freeQoS reads them but never puts a limit there
PPPoE	A connection where the subscriber logs in with a username. During the session, RouterOS creates a <code><pppoe-login></code> interface for that subscriber alone.	That interface carries the subscriber's byte counters
CPE	The equipment at the subscriber's premises (antenna, box)	Its MAC address links the subscriber to its radio sector
Backhaul, sector	A radio link carrying the traffic of a site; a sector is an antenna serving several subscribers	Their capacity varies (weather, interference): they are often the real bottleneck
Rate / volume	A rate is a speed (Mbit/s), a volume a quantity (bytes)	Rates drive shaping, volumes drive billing
Latency, RTT	The round-trip time of a packet, in milliseconds	It is what makes a video call or a game good or bad
Queue	Where packets wait when a link is full	The longer the queue, the higher the latency
Shaping	Deliberately limiting a rate by holding packets in a queue we control	So that the queue forms where it is well managed
CAKE	A modern queueing algorithm built into RouterOS 7	It limits the rate without letting latency explode
Bufferbloat	The latency added by oversized queues when a link saturates	The first cause of a bad experience on a loaded network
QoS / QoE	QoS: the network mechanisms. QoE: what the subscriber feels.	freeQoS tunes QoS to improve QoE, and scores QoE
VRF	A separate routing table on the same router	The ping must leave in the right table, or it takes a detour
DSCP	A priority label in the header of every IP packet (46 = voice)	The measurement ping carries it to jump the queue
NetFlow	A summary of conversations that the router sends to a collector	It tells what, where to, and how much, without copying the traffic

Notion	What it is	Why freeQoS cares
RouterOS API	The programming interface of MikroTik routers	The only way freeQoS reads and writes
TimescaleDB	A PostgreSQL extension for time series	It stores millions of measurements and purges old ones

2. Architecture

freeQoS is **out-of-band**: it is never in the traffic path. It reads the routers and the antennas, stores what it measures, and installs CAKE queues **on the routers**. The routers are the ones limiting traffic. If freeQoS stops, the queues stay in place and traffic flows normally; only measurement, timed boosts and adjustments pause.

Everything is read continuously; a single path writes to the routers



Architecture: sources, cycles, database, and the single write path

Every grey arrow is a read. The coloured arrow is the only write path: it starts at the planner and goes through the safeguards of section 7.

Components

Component	Technology	Role
Application	Python, FastAPI, a single process	API, interface, scheduler, collectors

Component	Technology	Role
Scheduler	Internal asynchronous loop	Runs each cycle at its period; a slow cycle does not block the others
RouterOS client	<code>librouteros</code> , API 8728 / 8729	Reads and writes; one connection per router, with a timeout
NetFlow collector	UDP listener in the same process	Decodes v5, v9 and IPFIX, aggregates in memory
Database	PostgreSQL with TimescaleDB	Time series (hypertables) and reference data
Interface	Framework-free HTML and JavaScript, served by the application	Operating pages

A measurement, from the router to the screen

1. The `collect_subscribers` cycle reads `/ppp/active` and the interface counters of each PoP, every 10 s.
2. The difference between two reads gives a rate. Impossible values are discarded (a counter going backwards, a rate above 100 Gbit/s).
3. The rate is written to a hypertable, together with the last RTT measured by `probe_rtt`.
4. Bufferbloat and the QoE score are computed when read, over the requested window. No score is stored in advance: a change of scale applies to the whole history.

A write, from the plan to the queue

1. A plan arrives (API, *Plans* page) or a capacity changes (radio, QoE loop).
2. The planner computes the desired state of each router: queue types, parent queues, subscriber queues.
3. It reads the actual state and produces a **plan**: the list of commands that would move from one to the other.
4. The safeguards filter the plan: ownership, transit, duplicates, oscillation, circuit breaker.
5. If writing is enabled, the plan is sent command by command, parents first, and each command is logged. Otherwise it is only displayed.

An API call that changes a plan runs these five steps **immediately** for that subscriber. The 2-minute reconciliation catches up with everything else (PPPoE reconnection, new address, queue edited by hand).

Instance identity

Each installation has an identifier (`data/instance.id` , or `FREEQOS_INSTANCE_ID`). It is written into the comment of everything it creates: `freeqos:managed instance=<id>` . This is what lets two installations, or freeQoS and an operator, never overwrite each other's work.

3. Installation and updates

One command installs everything: `install.sh` sets up Docker if needed, writes a `.env` with random secrets, builds and starts the stack, then waits for the application to answer. Running the same script again updates while keeping the data.

Required resources

The figures below were **measured** on 8 October 2026 on a virtual machine with 4 vCPU and 16 GB of RAM: the real freeQoS application, the `timescale/timescaledb:latest-pg16` image, synthetic data. They give orders of magnitude; your network will refine them.

What consumes resources

Component	Idle	Under load	Note
Application (RAM)	75 MB	190 MB at 5,000 NetFlow flows/s; 575 MB after 15,000 flows/s	Memory taken at peak is not given back: size for the peak
Database (RAM)	190 MB	Grows with use	The TimescaleDB image sets its own cache to 25% of the machine's RAM (4 GB out of 16 GB)
Database journal (WAL)	512 MB	—	Fixed disk space
Docker images	2.8 GB	—	TimescaleDB 2.5 GB, application 0.3 GB

The NetFlow collector uses a single core. This is the real sizing limit:

NetFlow flows received per second	CPU (one core)	Loss
1,000	6%	0%
5,000	45%	0%
10,000	77%	3%
15,000	98%	17%

Beyond about 5,000 flows per second, datagrams are lost. Adding cores changes nothing; a faster core helps a little. To reduce the volume: export only from the internet edge (one vantage point instead of two), or enable sampling on the routers.

The number of flows per second depends on usage and cannot be guessed. To measure it, read `flows_seen` in `GET /api/v1/netflow/status` one minute apart, at peak time.

Disk

Measured per subscriber connected around the clock:

Table	Per day, raw	After compression (7 days)	Over 90 days
Subscriber measurements (<code>subscriber_metrics</code> , every 10 s)	1.2 MB	0.08 MB (6.5% of raw)	≈ 15 MB
NetFlow (<code>flow_metrics</code> + <code>flow_app_metrics</code> , per minute)	up to 0.83 MB	no compression	up to 75 MB

The NetFlow figure is a ceiling: it assumes traffic every minute, day and night, across three usage families. A real subscriber produces less. The two NetFlow tables are not compressed today, only purged after 90 days: they are the largest disk item.

How TimescaleDB stores measurements. A *hypertable* is split into one-day chunks. After 7 days, each chunk is rewritten as compressed columns: neighbouring values of a column look alike and compress very well. After 90 days, the whole chunk is dropped at once, without scanning the table.

Recommended sizing

Network	vCPU	RAM	SSD disk
Lab, demonstration (≤ 100 subscribers)	2	4 GB	40 GB
≤ 500 subscribers	4	8 GB	80 GB
≤ 1,000 subscribers	4	8 GB	150 GB
≤ 5,000 subscribers	8	16 GB	600 GB

These values keep a 50% disk margin over the measured worst case (≈ 90 MB per subscriber over 90 days), plus the images and the journal. They assume the NetFlow volume stays under 5,000 flows/s; otherwise, see above. An SSD is required: the database writes continuously.

On the routers, freeQoS adds:

- an API read every 10 s (sessions and counters);
- 20 subscribers × 5 pings every 30 s per PoP;
- the NetFlow export, which uses router CPU.

Watch the routers' CPU load in *Devices* after commissioning.

Prerequisites

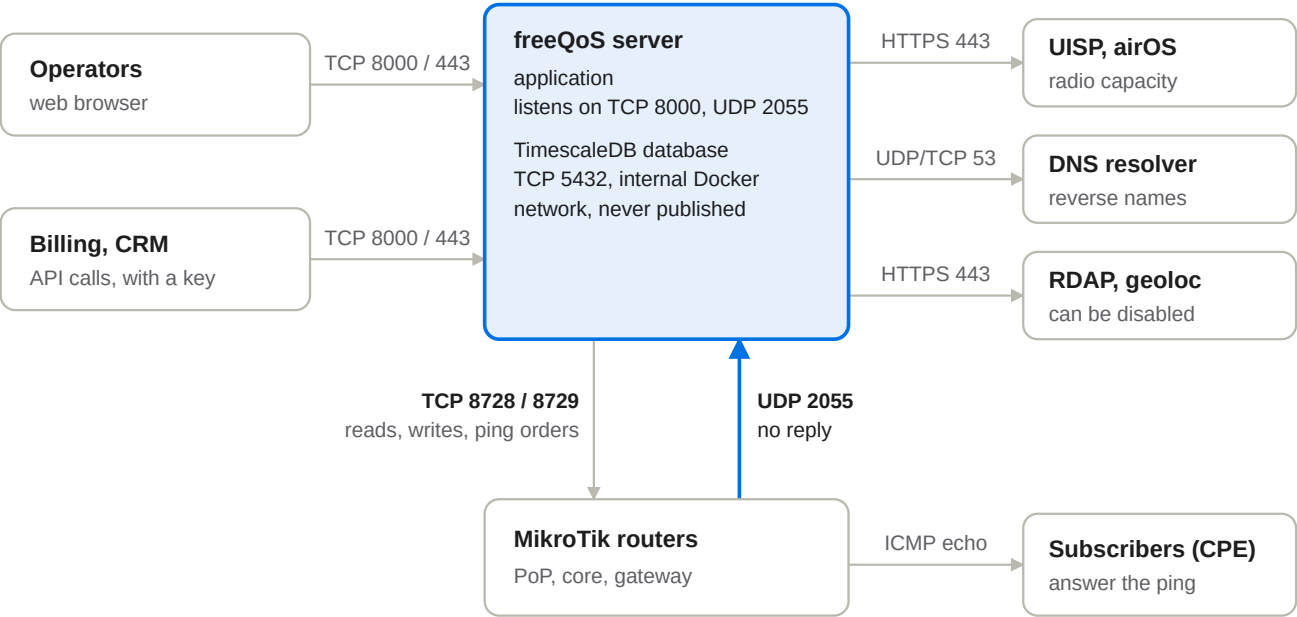
Item	Requirement
System	64-bit Linux. The script installs Docker and <code>git</code> itself on Debian, Ubuntu, RHEL, Rocky or Alma (<code>apt</code> or <code>dnf</code>).

Item	Requirement
Rights	<code>root</code> or <code>sudo</code>
Address	A fixed IP, reachable from the routers (for NetFlow) and able to reach them (for the API)
Internet access during installation	<code>github.com</code> , <code>get.docker.com</code> , Docker Hub or <code>mirror.gcr.io</code> , <code>pypi.org</code>
Routers	RouterOS 7, a dedicated account (step 3 below)
Optional	A DNS name and an HTTPS proxy in front of the interface

Network flows

Only one connection arrives without a reply: the routers' NetFlow

The arrow starts from whoever opens the connection; the reply comes back over the same connection.



Network flows: who opens which connection, on which port

A stateful firewall only needs a rule in the direction the connection is opened: the reply is accepted as part of the connection. The only exception is NetFlow: a one-way UDP send, from the router to freeQoS, with no reply.

#	From → to	Protocol, port	Content	Needed
1	Browser → freeQoS	TCP 8000 (or 443 behind a proxy)	Interface	Yes
2	Billing, CRM → freeQoS	TCP 8000 or 443	API, with a key	If integrated
3	freeQoS → routers	TCP 8728 (clear) or 8729 (TLS)	Reads, queue writes, ping orders	Yes

#	From → to	Protocol, port	Content	Needed
4	Routers → freeQoS	UDP 2055, no reply	NetFlow	For traffic and <code>/usage/v1</code>
5	Router → subscribers	ICMP <i>echo request</i> , then the subscriber's <i>echo reply</i>	Latency probe; does not go through freeQoS	For latency
6	freeQoS → UISP controller	HTTPS 443	Capacity of the radio links	If <code>BACKHAUL_PROVIDER=uisp</code>
7	freeQoS → Ubiquiti antennas	HTTPS 443 (<code>/status.cgi</code>)	Radio capacity, read on each antenna	If <code>BACKHAUL_PROVIDER=airos</code>
8	freeQoS → DNS resolver	UDP and TCP 53	Reverse names of the addresses seen	Recommended
9	freeQoS → <code>rdap.org</code>	HTTPS 443	Owner of the addresses	Enabled by default, can be disabled
10	freeQoS → <code>ipapi.co</code> , <code>ipwho.is</code> , <code>freeipapi.com</code> (HTTPS 443), <code>ip-api.com</code> (HTTP 80)		Geolocation	Enabled by default, can be disabled
11	Application → database	TCP 5432, internal Docker network	Reads and writes of measurements	Never published outside the server
12	freeQoS → FreeRADIUS database	Port of the SQL database	Plans read from RADIUS	If <code>PLAN_PROVIDER=freeradius_sql</code>
13	Server → internet	HTTPS 443	Installation and updates	During installation

On the router, the firewall's `input` chain must accept TCP 8728 or 8729 from the freeQoS address. The NetFlow export leaves through the `output` chain, which is usually open.

Step-by-step installation

- 1. Prepare the server.** A virtual machine sized according to the table above, with a fixed IP and an SSD.
- 2. Open flows 1, 3, 4 and 13** of the table, plus 2 if a billing system calls the API.
- 3. Create the freeQoS account on each router.** In the RouterOS terminal:

```
/user/group add name=freqos policy=read,write,api,test
/user add name=freqos group=freqos password=<strong-password> address=<freeQoS-IP>/32
/ip/service set api address=<freeQoS-IP>/32
```

`address=` restricts the account and the API service to the freeQoS address. For the encrypted API, enable `api-ssl` (port 8729) with a certificate, and tick SSL when adding the router. To start read-only, remove `write` from the group: freeQoS will measure without writing anything.

4. Install freeQoS on the server:

```
curl -fsSL https://raw.githubusercontent.com/tatdorian/freeQoS-/HEAD/install.sh | sudo sh
```

If the repository is private, this link does not answer without authentication. Clone the repository with an access token instead, then run `sudo ./install.sh` from its folder.

Without asking anything, the script:

1. installs Docker, the `docker compose` plugin and `git` if they are missing;
2. fetches the code into `/opt/freeqos` (main branch, or `FREEQOS_BRANCH`);
3. writes `.env` : random PostgreSQL password, `APP_PORT=8000` , `NETFLOW_PORT=2055` ;
4. switches to `mirror.gcr.io` if Docker Hub is unreachable;
5. builds, starts, waits for `/health` (3 minutes at most) and prints the address.

5. Check.

```
cd /opt/freeqos && sudo docker compose ps
curl -s http://127.0.0.1:8000/health/ready
```

Both containers `freeqos-app` and `freeqos-db` must be `running` , and `/health/ready` must answer `"status":"ready"` .

6. Create the first account. Open `http://<freeQoS-IP>:8000` . The first visit offers to create the administrator account (12 characters minimum).

7. Put the interface behind HTTPS (recommended): a reverse proxy in front of port 8000. With Caddy, a few lines are enough, and the certificate is obtained automatically:

```
<freeqos-dns-name> {
    reverse_proxy 127.0.0.1:8000
}
```

Then close port 8000 to the outside on the firewall.

8. Add the routers. *Devices* → *Connect a router*: name, IP, the account from step 3, role (`pop` , `core` or `gateway`). *Test* checks the connection without saving anything. After adding, the automatic commissioning described in section 4 runs; follow its report.

9. Let it run in simulation, for at least one full cycle. Check the write plan in *Settings*: planned queues, skipped links and their reason. Then enable writing.

10. Schedule the backup. For example, once a night:

```
0 3 * * * cd /opt/freeqos && make backup
```

Copy the `backups/` folder off the server regularly. `make backup` does not include the instance identifier: save it once with `sudo docker compose exec -T app cat /app/data/instance.id > instance.id`, or set it with `FREEQOS_INSTANCE_ID` in `.env`.

What must be kept

File	Content	If lost
<code>.env</code>	Database password, ports	The database becomes unreachable.
<code>data/secret.key</code>	Encryption key for router passwords	Router passwords stored in the database become unreadable; they must be entered again.
<code>data/instance.id</code>	Identifier of this instance	Queues created earlier become those of "another instance" and are no longer touched.
<code>pgdata</code> volume	All measurements, the inventory, the plans	History lost.

Backup and restore: `make backup` and `make restore` (with the `timescaledb_pre_restore` / `post_restore` steps).

Kubernetes

freeQoS recognises a pod (`KUBERNETES_SERVICE_HOST` , `/var/run/secrets/kubernetes.io`). Three things to set:

- **NETFLOW_COLLECTOR_ADDRESS** : the address the routers must send to (UDP Service, LoadBalancer or node). Without it, freeQoS sets no NetFlow target rather than announcing the pod IP.
- **A persistent volume** for `data/` (key and instance identifier).
- **FREEQOS_INSTANCE_ID** for a readable, stable identifier.

RouterOS prerequisites

Item	Value
API service	<code>api</code> (8728) or <code>api-ssl</code> (8729) reachable from freeQoS
Account	A group with the policies <code>read,write,api,test</code> (<code>test</code> is used by the ping probe)
NetFlow export	Set up by freeQoS itself; UDP port 2055 must be reachable from the routers

Item	Value
Version	RouterOS 7 recommended (VRF, <code>ping vrf=</code> , CAKE)

Updating

1. If two instances drive the same routers, stop one **before** updating.
2. Run `install.sh` again (or `git pull` then `docker compose up -d --build`).
3. On the first start of a new version, database migrations apply by themselves; the database keeps its data.

4. Routers and inventory

freeQoS only knows the routers it is given. Everything else (subscribers, links, tree, antennas) is discovered from them.

How freeQoS talks to a router

freeQoS uses a single channel: the **RouterOS API**. No SSH, no SNMP, no script installed on the router.

The protocol. The API is a TCP connection on port 8728 (clear) or 8729 (TLS-encrypted, `api-ssl` service). It carries the same commands as the router's terminal, as words: `/ppp/active/print`, `/queue/simple/add`, `/ping`. The router answers with lines of `key=value` fields, one per object. freeQoS therefore reads exactly what a technician would see in the terminal.

The rights. The RouterOS account belongs to a group, and the group grants *policies*:

Policy	What it allows	Used for
<code>api</code>	Connecting through the API	Everything
<code>read</code>	Reading configuration and state	Sessions, counters, routes, neighbours
<code>write</code>	Changing the configuration	Queues, queue types, NetFlow export, restrictions
<code>test</code>	Running <code>/ping</code> and <code>/tool/traceroute</code>	The latency probe

Without `write`, freeQoS runs read-only: measurement, latency and NetFlow (if the export was set up by hand) work, shaping does not.

Failures. Each request has a timeout (`timeout_s`, 5 s by default). Routers are queried in parallel: a router that does not answer misses its read and reports it, the others are read normally.

Declaring a router

Two sources add up:

- **The database**, through the interface or `POST /api/v1/pops/routers`. This is the normal case; the password is encrypted with `data/secret.key` and never comes out of any response.
- **A file** (`ROUTERS_FILE`, YAML or JSON) or the `ROUTERS=' [ {...} ] '` variable. Useful for declarative deployment; a router from the file can be hidden from the interface without touching the file.

Field	Default	Role
<code>name</code>	—	Unique identifier of the PoP
<code>host</code>	—	Management IP or name
<code>port</code>	8728	8729 for <code>api-ssl</code>
<code>username</code> / <code>password</code>	<code>qos-ro</code>	RouterOS account

Field	Default	Role
<code>role</code>	<code>pop</code>	<code>pop</code> , <code>core</code> or <code>gateway</code> ; a core or gateway is never treated as a subscriber
<code>use_ssl</code> , <code>tls_verify</code>	<code>false</code> , <code>strict</code>	<code>strict</code> , <code>fingerprint</code> (with a SHA-256 <code>tls_fingerprint</code>) or <code>insecure</code>
<code>loopback</code>	inferred	Identity of the router in the tree (a /32 on a <code>lo*</code> interface, or the router-id)
<code>timeout_s</code>	5	Between 0.5 and 60 s
<code>pppoe_interface_pattern</code>	<code><pppoe-{login}></code>	Name of a subscriber's dynamic interface

`POST /api/v1/pops/routers/test` tries a connection without saving anything; on failure, the response gives a hint (port closed, wrong credentials, missing `api` policy, certificate).

Automatic commissioning

As soon as a router is added, freeQoS immediately runs, in order, the cycles that would otherwise run at their own pace. Each step has 90 s at most.

1. Check the account's actual write rights.
2. Read the PPPoE sessions.
3. Load the plans.
4. Read the ports.
5. Discover the topology (tree, uplink, loopback).
6. Create the CAKE queue types and the subscriber queues.
7. Set up the NetFlow export.
8. Apply the traffic restrictions.
9. Read the DNS cache.
10. Measure latency.

The report (`GET /api/v1/pops/provisioning/{router}`) then reads the router back and says what is **actually** in place. If writing is disabled or the account lacks `write` , the state becomes `blocked` , with the action that unblocks it.

Who is a subscriber on this PoP?

RouterOS has no "subscribers" table. The census therefore starts from the subnets the router serves (`/ip/address`). Each subnet is classified: point-to-point, routing transit, PPPoE server or **subscriber**.

Seven sources are then cross-checked; none can erase what another one saw.

Source	What it reveals
<code>/ppp/active</code>	PPPoE subscribers

Source	What it reveals
ARP	Static-IP subscribers (the entry disappears after a few minutes of silence)
DHCP leases	DHCP subscribers, even silent ones
Bridge host table	Layer-2 presence behind a VLAN-filtering bridge
Static routes	Routed blocks (/29...) that never appear in ARP
Existing queues	Subscribers already managed
MNDP / LLDP	Neighbouring equipment

The result also states what it does not know: unreadable source, empty subscriber subnet, address outside any known subnet.

Topology

The tree is built from neighbours, shared subnets, tunnels, MAC addresses and, if configured, UISP. It is refreshed every 15 minutes. Everything can be corrected by hand and the correction survives rediscovery: move a box, change a parent, merge two boxes that are the same device, create or delete a link, correct a role.

How the tree is discovered

No single source is enough. freeQoS cross-checks seven:

Source	What it teaches
<code>/ip/neighbor</code> (MNDP, LLDP, CDP)	For each port, the device on the other side: name, model, MAC, IP. This is physical adjacency. MikroTik equipment and most radios announce themselves this way to their direct neighbours.
<code>/interface/ethernet</code>	The negotiated speed of each port: the physical ceiling of the link
<code>/ip/address</code>	The IP network of each interface, hence who shares a segment with whom
UISP	The radio links and their current real capacity
<code>/ppp/active</code> , <code>caller-id</code> field	The MAC of each PPPoE subscriber's CPE
<code>/ip/arp</code>	Addresses present on routed VLANs without PPPoE (candidates, never subscribers)
The configuration (<code>/ip/route</code> , OSPF, BGP, VLAN, bridges)	What the routers do with traffic: this is what gives the hierarchy

Three rules make the tree reliable:

1. **A router is identified by its loopback**, not by its name or an interface address. Two sites configured from the same template often have the same /30 link; only the loopback is unique.
2. **The hierarchy comes from the declared role** (gateway, core, PoP), not from a guess based on the hardware model.

3. **A subscriber's sector comes from a join:** the MAC read in `caller-id` is matched against the stations known to UISP. It is the only way to know which antenna a subscriber goes through, hence its real chain of bottlenecks. Without UISP, only the PoP is known.

Static-IP subscribers

A subscriber without PPPoE (fixed IP, routed block, VLAN) is declared in *Static clients*: a name, one or more addresses or networks, a PoP. It is then measured, limited and scored like a PPPoE subscriber.

5. Measurements: throughput, latency, QoE

Throughput

Byte counters are read through the RouterOS API, then turned into rates by the difference between two reads.

Object	Source	Period
PPPoE subscriber	Dynamic interface <code><pppoe-login></code>	10 s
Static-IP subscriber	Its own queue	10 s
Port, link	<code>/interface</code>	10 s
Radio backhaul	UISP or airOS (announced capacity)	30 s

Two safeguards reject absurd values. A counter going backwards (restarted session) does not produce a spike. A rate above `MAX_PLAUSIBLE_BPS` (100 Gbit/s) is discarded.

How a rate is computed

A router never gives a rate. It keeps **counters**: the total number of bytes received (`rx-byte`) and sent (`tx-byte`) by each interface since it was created. freeQoS reads these counters at regular intervals and divides the difference by the elapsed time.

Example: at 10:00:00 the counter reads 5,000,000,000 bytes; at 10:00:10, 5,012,500,000.
Difference: 12,500,000 bytes, i.e. 100,000,000 bits, in 10 s → **10 Mbit/s**.

Three details matter:

- **Where a PPPoE subscriber's counters are.** `/ppp/active` gives the login, the address and the session uptime, but not the bytes. Those are on the dynamic interface `<pppoe-login>`. freeQoS makes two reads and links them by name.
- **Direction.** The router counts from its own point of view: what it **receives** from the subscriber (`rx`) is the subscriber's **upload**, what it **sends** to it (`tx`) is the subscriber's **download**. freeQoS swaps the two.
- **Reconnection.** When a PPPoE session restarts, the interface is recreated and its counters start from zero. If the session uptime goes backwards or a counter decreases, freeQoS writes **no** rate for that cycle. A gap in the curve is visible; a fake spike would distort averages and scores.

A static-IP subscriber has no interface of its own: its traffic mixes with that of a VLAN or a port. Its bytes are read from the counters of **its queue** (`/queue/simple` , `bytes` field). Consequence: a static-IP subscriber is only measured once a queue targets it.

Latency: the probe

A subscriber's latency is the round-trip time measured **from its PoP** to its address. It goes neither through the core nor through freeQoS: the router does the pinging.

How a ping measures latency

The router sends a small ICMP *echo request* packet to the subscriber's address. The subscriber's CPE answers with an *echo reply*. The time between sending and the reply is the **RTT** (round-trip time).

A single ping means nothing: one packet may wait behind a download, another may go straight through. freeQoS therefore sends 5, 200 ms apart, and keeps:

- the **median** (the value displayed): it ignores a single very slow ping;
- the minimum and the maximum;
- the **jitter**: the variation from one ping to the next, which hurts voice;
- the **loss**: the share of pings left unanswered.

Why from the PoP. The ping leaves from the router serving the subscriber, not from the freeQoS server. It therefore measures the last segment, the one the operator controls and that decides the experience: radio, CPE, queue. A ping from freeQoS would add the core and the management network, unrelated to what the subscriber experiences.

Why a round-robin. Pinging 2,000 subscribers every 30 s would load the routers. Each cycle takes 20 subscribers per PoP, and the next one resumes where the previous one stopped. PoPs are probed in parallel.

Parameter	Value	Setting
Cycle	30 s, all PoPs in parallel	RTT_INTERVAL_S
Batch	20 subscribers per cycle and per PoP, round-robin	RTT_BATCH_SIZE
Pings	5, 200 ms apart	RTT_COUNT , RTT_PING_INTERVAL_MS
Priority	DSCP 46 (EF)	RTT_PROBE_DSCP (0 = disabled)
Freshness	A measurement older than 5 min is no longer displayed	RTT_MAX_AGE_S

Three choices make the measurement faithful.

VRF, in two sentences. A VRF is a separate routing table on the same router: subscriber routes can live in `CUST-INET` while the `main` table serves management. A packet only sees the routes of the table it leaves in.

DSCP, in two sentences. Every IP packet carries in its header a 6-bit label stating its class of service. The value 46 (*Expedited Forwarding*) means "voice, send first", and queues that honour it serve it before the rest.

1. Ping in the subscriber's VRF. If subscriber routes live in a VRF (for example `CUST-INET`), a ping without a table leaves through the `main` table, goes up to the core and comes back: 700 to 900 ms instead of 2 ms. freeQoS therefore picks the table from the most specific route to the subscriber:

1. a connected route first;
2. otherwise a route that does not point to the upstream gateway;
3. a VRF is preferred over `main`.

The parameter sent is `vrf=` , falling back to `routing-table=` for versions that do not know it.

2. Priority ping. The probe's packets carry DSCP 46 (EF). CAKE puts them in the *Voice* tin, with its default setting (`diffserv3`) as with `diffserv4` . They do not queue behind the subscriber's traffic: the latency measured is that of the path, not that of a subscriber saturating its plan. Only `CAKE_DIFFSERV=besteffort` cancels this effect. If the router refuses the DSCP parameter, the ping is sent again without it.

3. Slow fallback. If the median exceeds the interval between pings with at least 50% loss, the subscriber is re-tested with 3 pings 1 s apart for 10 minutes. This avoids counting merely late pings as lost.

When latency stays empty

A subscriber that has never answered is not a degraded subscriber: its CPE often filters ICMP. It is flagged `ever_answered=false` , excluded from the score and from the QoE loop, and the interface explains why. The **Find the cause** button (`GET /api/v1/rtt/diagnose`) replays the probe by hand and returns a verdict:

Code	Meaning	Action
<code>ok</code>	The subscriber answers now	The earlier silence was temporary
<code>no_test_policy</code>	The account is not allowed to run <code>/ping</code>	Add the <code>test</code> policy to the group
<code>rate_limited</code>	The subscriber answers one ping per second but drops a burst	None: the probe switches to the slow pace on this router by itself
<code>loopback_return_path</code>	The subscriber answers, but not to the router's loopback address	None: the probe now pings without a source
<code>no_route</code>	The router has no route to this subscriber	Check that the subscriber is really behind this PoP
<code>firewall</code>	Router rules drop ICMP before accepting established connections (rules listed)	Move an <code>accept icmp</code> rule up
<code>client_blocks_icmp</code>	The gateway answers, the subscriber never does	Allow ICMP echo on the CPE's WAN side
<code>router_cannot_ping</code>	The router gets no reply from its own gateway either	Check the router's output/input firewall

The response also contains `routing_table` , the `route_candidates` and a control ping to the gateway.

Bufferbloat

Bufferbloat is the latency that load **adds**. freeQoS computes it by crossing the RTT and the rate of the same sample: latency when the link is loaded, minus latency when it is not. No dedicated test is run.

Why bufferbloat exists

Every device (CPE, radio, router) has a buffer to absorb bursts. When the link is full, packets pile up in it and wait their turn. Manufacturers size these buffers for throughput, not for latency.

Example: a 1 MB buffer on a 10 Mbit/s link. When full, it holds 8 Mbit, i.e. **0.8 s** of waiting for the last packet in. A video call whose packets sit behind a download takes those 800 ms.

An idle latency test does not see it: the buffer is only full under load.

How freeQoS computes it

Each collection cycle writes one sample per subscriber: its rate and its last RTT. Over the requested window:

1. **Idle latency** = the 20th percentile of **all** RTTs. This is the observed floor, without being fooled by a single low value.
2. **Loaded slice** = the samples whose rate is at least the 60th percentile of the subscriber's rates. If the subscriber's rate is constant, the cut is at the median.
3. **Loaded latency** = the 90th percentile of the RTTs in the loaded slice.
4. **Bufferbloat** = loaded latency – idle latency, never negative.

At least 4 samples are needed, including 2 in the loaded slice. Otherwise freeQoS gives **no** grade: showing A+ for a subscriber who never downloaded anything would be a false good result.

Grade	Added latency	Colour
A+	≤ 5 ms	green
A	≤ 30 ms	green
B	≤ 60 ms	amber
C	≤ 100 ms	amber
D	≤ 200 ms	red
F	> 200 ms	red

QoE score (0 to 100)

The score is the **minimum** of two components:

- **Idle latency:** $100 - 0.6 \times (\text{RTT} - 10 \text{ ms})$. Below 10 ms, no penalty.
- **Bufferbloat:** interpolation between the points 0 ms $\rightarrow$ 100, 5 $\rightarrow$ 90, 30 $\rightarrow$ 80, 60 $\rightarrow$ 65, 100 $\rightarrow$ 50, 200 $\rightarrow$ 25 and 400 $\rightarrow$ 0.

The weakest link sets the score. A satellite link at 600 ms stays bad even without bufferbloat; a link at 8 ms that climbs to 400 ms under load is not excellent.

Each score states its basis (**basis**):

- **composite** : both components;
- **load** : bufferbloat only;
- **latency** : RTT only, a mere substitute when no load can be correlated.

Display thresholds: ≥ 80 green, ≥ 50 amber, below that red.

"At plan limit"

A subscriber filling its plan has high latency by construction. When the probe is not prioritised, its samples at the limit are set aside and the *At plan limit* badge explains it. When the probe is prioritised (DSCP 46 and CAKE), the measurement stays accurate even at the limit: nothing is set aside.

6. NetFlow traffic

Counters say **how much** a subscriber consumes. NetFlow says **what** and **where to**.

How NetFlow works

The problem. There are two ways to know what traffic is made of. The first is to copy every packet to a probe (*port mirroring*): on a 10 Gbit/s internet edge, that is 10 Gbit/s more to carry. The second is to ask the router to **summarise** what it sees. That is NetFlow: a few tens of kbit/s to describe thousands of conversations.

A flow. For NetFlow, a conversation is a *flow*: all packets with the same source address, the same destination address, the same ports and the same protocol. A video call gives a few flows; a web page, a few dozen.

The cache. The router keeps one line per ongoing flow in memory. For each packet, it adds its bytes to the flow's line. It sends nothing until the flow has *expired*:

- **inactive timeout:** no packet for 15 s, the flow is considered finished and exported;
- **active timeout:** a flow still in progress is exported anyway every minute, then its count starts again from zero.

RouterOS's default active timeout is **30 minutes**: an hour of streaming would show up as two records, half an hour late. freeQoS therefore sets `active-flow-timeout=1m` and `inactive-flow-timeout=15s` on each router.

The export. Expired flows leave in UDP datagrams towards the collector. UDP resends nothing: a datagram that does not arrive is lost for good. This is why the collector listens permanently and does no slow processing while receiving.

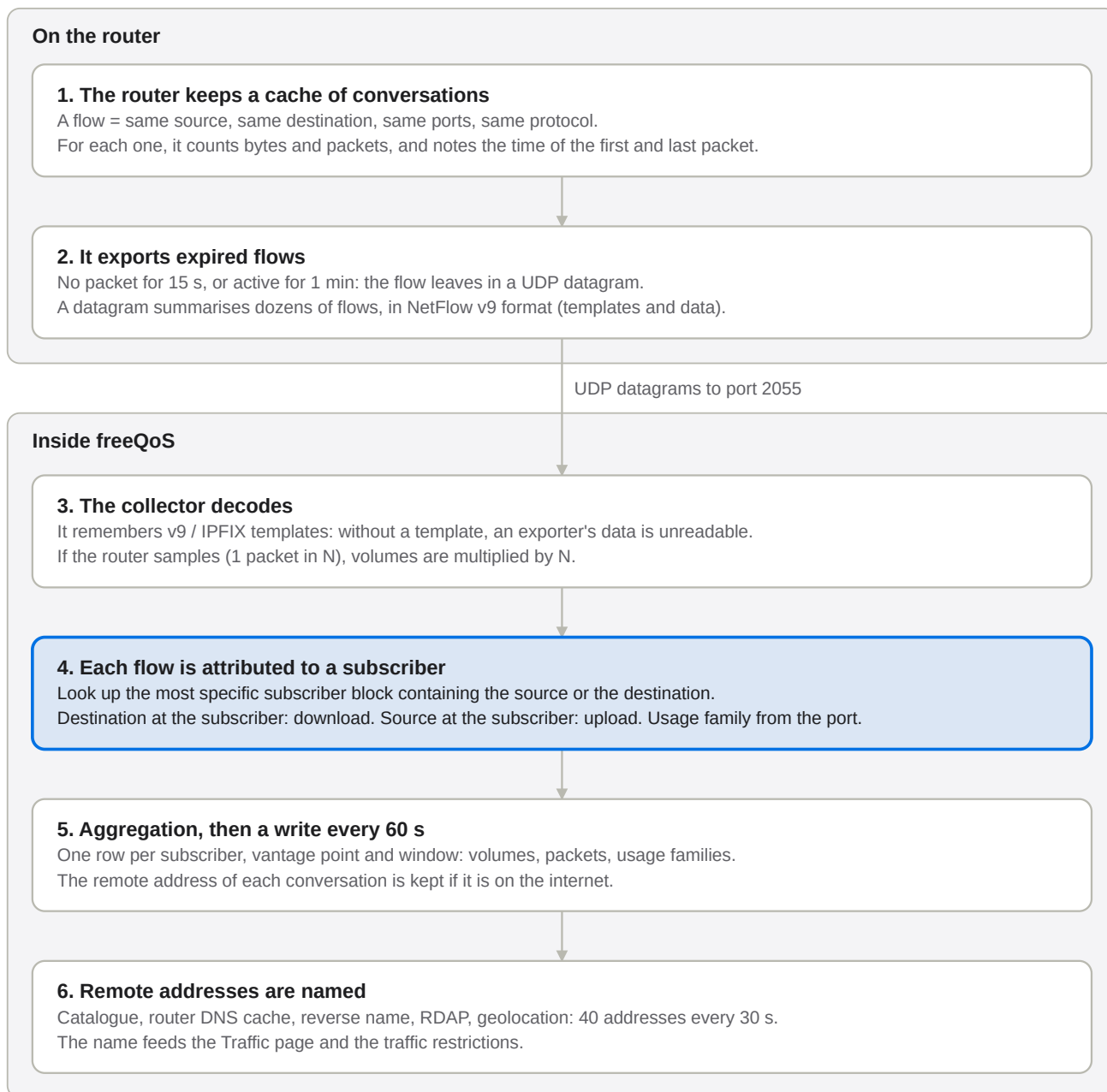
The formats.

Version	Principle	Particularity
v5	Fixed format, IPv4 only	Readable immediately
v9 (default set by freeQoS)	The router first sends templates describing the fields, then data referring to them	IPv4 and IPv6; without the template, the data is unreadable
IPFIX (v10)	Standardised version of v9	Works the same way

In practice: after a freeQoS restart, v9 data is ignored until the router resends its templates, which it does regularly. A few minutes of traffic may be missing; this is normal and counted.

Sampling. A very busy router may only examine one packet in N. It announces it, and freeQoS multiplies volumes by N.

A flow goes from the router's cache to a subscriber's account in six steps



NetFlow: from the router's cache to a subscriber's account

Step 4 is the heart of the collector: it turns "45 MB between two addresses" into "45 MB downloaded by this subscriber".

Collector

freeQoS listens on UDP port 2055 (`NETFLOW_PORT`) and decodes NetFlow v5, v9 and IPFIX. Flows are aggregated in memory then written every 60 s (`NETFLOW_FLUSH_INTERVAL_S`).

Four counting rules:

- 1. Direction comes from the subscriber, not the interface.** A flow whose destination falls in a subscriber's block is download for it; one whose source falls in it, upload. Rewiring therefore distorts nothing.

2. **The vantage point is part of the key.** The same byte crosses the PoP then the internet edge, and both export it. Counters are kept per vantage point, never added up.
3. **An unknown address is not a subscriber.** It goes into the list of "hosts seen", used only to help declare subscribers.
4. **The other end is kept.** The remote address of each flow is recorded to answer "who watches which service".

Attribution, in detail

freeQoS builds an **index of subscriber blocks**: each PPPoE subscriber's address (/32) and the networks of static-IP subscribers (/29, /30...). For each flow, it looks up the **most specific** block containing the source, then the destination. Blocks are sorted by prefix length: a lookup costs a few comparisons, whatever the number of subscribers. An internet edge sends tens of thousands of flows per minute; a slow lookup would lose datagrams.

Vantage points

The same byte crosses several routers, and each can export it. freeQoS distinguishes two vantage points:

Vantage point	Where	What it sees
edge	The internet edge, upstream of the core	Everything going to or coming from the internet, once
pop	The subscriber's PoP	The same traffic, plus local traffic, where VLAN and sector are known

Counters are kept per vantage point and are **never added up**. Consumption (Traffic page, `/usage/v1` API) is read from a single one: `NETFLOW_ACCOUNTING_VANTAGE`, `edge` by default. For the list of conversations, freeQoS keeps a single source per conversation: the one closest to the subscriber.

Rate of a flow

A record carries a volume and a duration (first → last packet). The rate is the volume divided by the **actual** duration. With RouterOS's 30-minute timeout, dividing by an assumed minute showed a rate thirty times too high; this is why the duration is always read from the record.

Usage families

Traffic is classified into broad families by **service port**: the smaller of the two ports, because the client picks its port at random above 32,768 and the server listens below.

Port	Family
80, 443, 8080, 8443	web
53	dns
25, 110, 143, 465, 587, 993, 995	email

Port	Family
3478, 5004, 5060, 5061	voice / video
3074, 27015	gaming
500, 1194, 1723, 4500, 51820, ESP/AH protocols	vpn
6881 to 6999	p2p
ICMP	diagnostics

Encryption sends Netflix, YouTube and the rest of the web through port 443: the port cannot tell them apart, and the family is honestly called "web". What distinguishes Netflix from YouTube is the **name** of the remote address (below).

What is left out of the list of conversations

- **Management traffic** (SNMP, BGP, BFD, RADIUS, syslog, NetFlow, Winbox, RouterOS API): the network administering itself, not a subscriber.
- **DNS queries** (ports 53, 853, 5353): asking 8.8.8.8 for a site's address is not "going to 8.8.8.8". Their volume is still counted.
- **Private addresses**: two subscribers talking to each other are not an internet destination for either of them.

At most 2,000 destinations are kept per 60-second window, so that a subscriber running p2p does not trigger a burst of writes. A destination not seen for 7 days is forgotten; its name is kept.

Export set up automatically

freeQoS configures the export on each router itself, every 10 minutes if needed (`NETFLOW_EXPORT_AUTO` , `NETFLOW_EXPORT_INTERVAL_S`):

```
/ip/traffic-flow set enabled=yes interfaces=all active-flow-timeout=1m inactive-flow-timeout=15s
/ip/traffic-flow/target add dst-address=<collector> port=2055 version=9
```

The collector address is not guessed. It is the local address the system would use to reach **this** router (a UDP socket opened towards it, without sending a packet). It is therefore correct even on a server with several interfaces.

Special cases:

- **Container (Docker, Podman, Kubernetes)**: the local address would be the container's, unreachable from the routers. Without `NETFLOW_COLLECTOR_ADDRESS` , freeQoS sets no target and says so.
- **Target towards another collector**: never touched. A third-party tool already receiving the flows keeps receiving them.
- **Stale target**: a target carrying the mark of **this** instance and pointing to an old address is removed.

Writing goes through the same path as queues: simulation by default, circuit breaker, audit.

Naming addresses

NetFlow only carries addresses. When a subscriber reaches an address never seen before, it is recorded "to be named". A separate loop then names it, in batches of 40 every 30 s, most recent first. Naming may need a network request, hence a wait: doing it while receiving would block the collector. An address reached ten seconds ago is named on the next pass.

Source, by order of confidence	What it gives	What it costs
Catalogue	Address blocks published by the large services (Netflix, YouTube, Meta...)	Nothing: works without internet, immediate answer
Router DNS cache	The name the subscriber asked for (<code>netflix.com</code>), following CNAMEs	Nothing. Only if subscribers resolve through the router; a subscriber querying 8.8.8.8 directly bypasses it.
Reverse name (PTR)	E.g. <code>ipv4-c001-par001.1.oca.nflxvideo.net</code> : follows a service that changes blocks	One DNS query per new address
RDAP (registry)	Organisation, AS number, country, announced block	One HTTP call to <code>rdap.org</code> per address
Geolocation	Country, region, city	One HTTP call to <code>ipapi.co</code> (then fallback services), or nothing with a local MaxMind database

An address without a reverse name is asked again 3 times at most. A name read from the DNS cache is no longer attributed after 24 h: the address may have changed owner.

A CDN is not streaming. Cloudflare, Akamai or Fastly serve a recipe site, a video or a software update alike. They are classified `cdn` , never `streaming` , so that a restriction placed on them is a conscious choice. The interface shows the source of each name (catalogue, reverse name, registry).

Privacy. RDAP and web-service geolocation are **enabled by default**. They send to third parties the addresses your subscribers visit. To avoid it:

- `IPFINDER_GEOIP_DB` points to a local MaxMind database (`.mmdb`): same result, no outgoing call;
- `IPFINDER_GEOIP_ENABLED=false` and `IPFINDER_RDAP_ENABLED=false` turn these two sources off (changeable live);
- the catalogue, the DNS cache and reverse names are enough to recognise the large services.

Traffic restrictions

A restriction targets a catalogue service for one or more subscribers. Two actions:

Action	What is installed
<code>block</code>	An <code>address-list</code> for the service and two <code>filter action=drop</code> rules (one per direction)
<code>limit</code>	An <code>address-list</code> , two <code>mangle</code> marks and two capped queues

Every 5 minutes (`RESTRICTIONS_INTERVAL_S`), the address list of each rule is recomputed and only the difference is pushed. A new Netflix address seen by NetFlow therefore joins the list by itself.

No rule is ever created automatically: throttling a service is a business decision. `GET /api/v1/traffic-rules/{id}/preview` shows what a rule targets today. IPv6 is not installed yet.

7. Plans and shaping

This is the only part of freeQoS that writes to the routers. It always follows the same path: **compute a plan, show it, apply it**, with several safeguards in between.

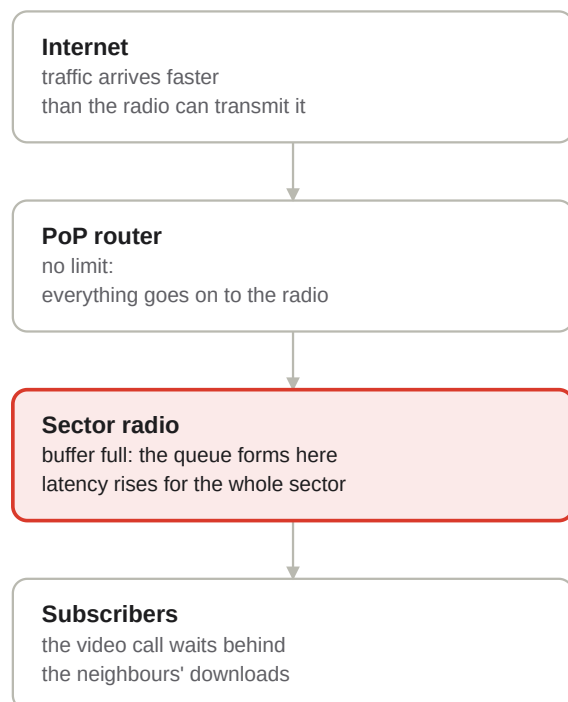
How shaping works

The queue always forms at the bottleneck. When a device receives faster than it can transmit, it stores the excess in a queue. That queue appears at the slowest point of the path: often a sector's radio or the subscriber's CPE. Their buffers are large and sort nothing: a video call waits there behind a download. This is the bufferbloat of section 5.

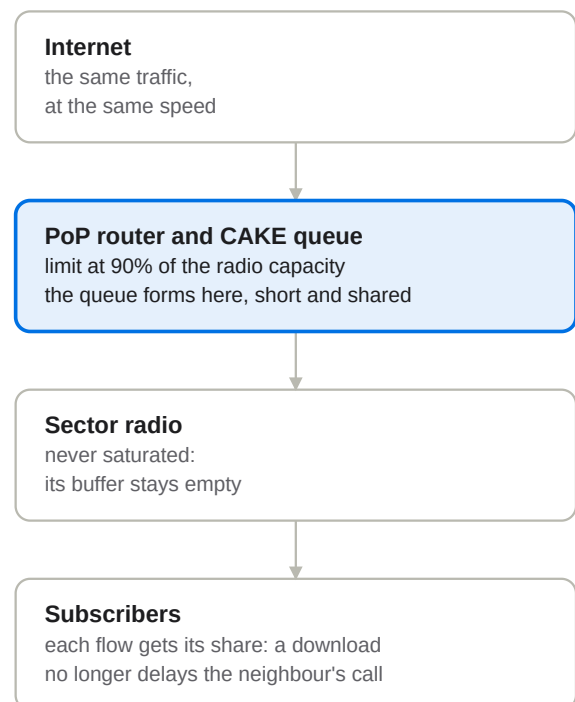
The idea of shaping. Traffic is deliberately limited **slightly below** the bottleneck capacity, at a place we control: the PoP router. The PoP then becomes the slowest point. The queue forms there, in a CAKE queue that keeps it short and shares it fairly. The radio's buffer stays empty. About 10% of throughput is given up (`SHAPING_SAFETY_FACTOR=0.90`), and latency stays low even when the sector is full.

Limiting slightly below capacity moves the queue to where it is well managed

Without freeQoS



With freeQoS



Shaping: where the queue forms, without and with freeQoS

How a limit acts on TCP. Most traffic (web, video, downloads) uses TCP, which speeds up as long as nothing is lost. When the subscriber exceeds its limit, the extra packets wait in the queue. If the wait lasts, CAKE drops one (or marks it, with ECN). The TCP sender understands it is going too fast and slows down. The rate thus settles on the limit, without a growing queue.

What CAKE does

CAKE (*Common Applications Kept Enhanced*) is the queueing algorithm freeQoS uses. It combines several mechanisms:

Mechanism	What it does	Effect for the subscriber
Flow isolation	Each conversation has its own small queue, served in turn	A download does not block a call
Triple isolation (<code>triple-isolate</code>)	Sharing happens first between hosts, then between the flows of a host	A host opening 100 connections does not get 100 shares
Active queue management (COBALT)	Measures the time each packet spends in the queue; if it stays too long, drops or marks a packet to slow the sender	The queue stays short: a few milliseconds instead of hundreds
Priority classes (<i>tins</i> , <code>diffserv</code>)	Reads the DSCP label and serves voice before the rest	Packets marked EF go first, including the latency probe
Overhead compensation (<code>overhead</code>)	Counts encapsulation bytes (PPPoE, VLAN) on top of each packet	The limit matches the real rate on the wire
NAT mode (<code>nat</code>)	Looks at addresses before translation	Fairness between hosts works behind CGNAT
Reference RTT (<code>rtt</code>)	Sets how fast queue management reacts	50 ms suits an access network

How queues nest

RouterOS queues form a tree:

- A sector's **parent queue** caps the **sum** of its subscribers' traffic at 90% of the link's measured capacity.
- **Each subscriber queue**, attached to that parent, caps its subscriber at its plan. It has two limits, `max-limit=upload/download` , and two CAKE queue types, one per direction.

A subscriber alone on a quiet sector gets its full plan. When everyone pulls at once, the parent queue shares the available capacity. If radio capacity drops (rain, interference), the parent queue drops on the next cycle, and the queue stays at the PoP.

Where a subscriber's plan comes from

The plan belongs to the subscriber, not to the PoP: a PoP has a capacity, not a plan. It is resolved in this order:

1. **The plan written for this subscriber**, pushed through the API (billing, Preseem integration) or entered on the *Plans* page. The last write wins.
2. **A subscriber pushed without a rate** gets the default plan (`DEFAULT_PLAN_DOWN_MBPS` / `DEFAULT_PLAN_UP_MBPS` , 100/20).
3. **A subscriber that is only detected** has no plan: it is observed, never throttled to a rate nobody sold, unless `DEFAULT_PLAN_FOR_DETECTED_CLIENTS=true` .

Deleting a subscriber's plan removes its queue on the next cycle.

What is installed on the router

Two queue types: `freeqos-cake-up` and `freeqos-cake-down`. Their CAKE options come from the settings:

Setting	Default	Role
<code>CAKE_OVERHEAD</code>	22	Encapsulation bytes counted per packet
<code>CAKE_RTT_MS</code>	50	Reference RTT for queue management
<code>CAKE_DIFFSERV</code>	RouterOS default	<code>besteffort</code> , <code>diffserv3</code> , <code>diffserv4</code> , <code>diffserv8</code> , <code>precedence</code>
<code>CAKE_FLOWMODE</code>	RouterOS default	Flow isolation (<code>triple-isolate</code> recommended)
<code>CAKE_NAT</code>	RouterOS default	Essential behind CGNAT
<code>CAKE_ACK_FILTER</code>	RouterOS default	Thins ACKs on a very asymmetric link
<code>CAKE_WASH</code> , <code>CAKE_MPU</code>	RouterOS default	Clearing DSCP on egress, minimum billed packet size

One parent queue per link (`freeqos-parent-<link>`): radio backhaul or sector. Its rate is `max(5 Mbit/s, measured capacity × 0.90)` (`SHAPING_SAFETY_FACTOR`, `SHAPING_FLOOR_MBPS`). The queue thus forms in CAKE, where it is controlled, rather than in the radio's buffer.

One queue per subscriber:

- **Target:** the subscriber's address (`target=10.20.0.10/32`), never the `<pppoe-...>` interface. The interface is recreated at each reconnection, and RouterOS reverses the direction of `max-limit` on it.
- **Parent:** the link whose network contains the subscriber's address; the most specific wins.
- **Write order:** parents first. RouterOS refuses a child whose parent does not exist.

Every object carries the comment `freeqos:managed instance=<id>`.

Reconciliation

Every 2 minutes (`SHAPING_RECONCILE_INTERVAL_S`), freeQoS reads the queues in place, computes the desired state and sends **only the difference**. A queue that is already right gets no command. Only the field that changes is rewritten: a plan change only touches `max-limit`.

Safeguards

Safeguard	Effect
Simulation by default	<code>ENFORCEMENT_ENABLED=false</code> at startup. Plans are computed and displayed, nothing is written. Writing is enabled in the interface, after reading the plan. If the variable is <code>false</code> in the environment, enabling is locked.
Ownership	Only objects marked by this instance are modified or deleted. A queue from another instance or created by hand is never touched, nor set to 0/0.

Safeguard	Effect
Transit links	No parent queue on a routing link, towards a managed router, towards the gateway or the core, or on the uplink. The reason is shown (<code>skip_reason</code>).
Duplicate targets	Two queues that would target the same address: neither is written and a planning error is reported.
Anti-oscillation	If a line's target, parent or rate returns to a previous value ($A \rightarrow B \rightarrow A$) within 1 h, the line is frozen and reported (<code>frozen_lines</code>). <i>Reset queues</i> unfreezes it.
Circuit breaker	A plan of more than 500 actions on one router (<code>ENFORCEMENT_MAX_ACTIONS</code>) is refused as a whole: it almost always betrays a badly computed desired state (empty inventory, lost capacity). Only <i>Reset queues</i> , explicitly requested, is sent in batches.
Audit	Every command sent is logged with its author (<code>GET /api/v1/shaping/audit</code>).

Manual adjustments

- **Rate override** (`PUT /api/v1/shaping/policies`): sets the rate of a link or a subscriber, above the computed one.
- **Boost** (`POST /api/v1/shaping/boosts`): a higher rate for a duration; it is removed when it expires (checked every 30 s).
- **Dry run** (`POST /api/v1/shaping/plan`): the commands that would be sent, without sending anything.

Closed QoE loop

Every 5 minutes, freeQoS looks at the QoE of each sector over the last 15 minutes.

- **Tighten**: if at least 2 subscribers of the sector fall below 55, the link's queue is tightened by 10%. The queue then forms again in CAKE rather than in the radio.
- **Floor**: it never goes below 50% of the computed rate.
- **Relax**: one step is given back after 3 healthy cycles in a row. Tighten fast, relax slowly, to avoid oscillation.
- **What never moves**: a subscriber's purchased plan. Only the sector's shared envelope changes.
- **What does not count**: a subscriber that has never answered the ping.

A single degraded subscriber points at its last mile (CPE, home Wi-Fi), not at the sector. This is why at least two are needed.

8. Interface

Each page reads in sections, with a table of contents at the top and a tooltip on every figure that deserves an explanation.

Page	Question it answers	Sections
Dashboard	What is happening now?	Total throughput, traffic per router, top consumers, radio backhauls
Executive	Where is the risk?	Saturation risks, latency per subscriber, load and queues per node, health over time (QoE heatmap)
Traffic	Who consumes what, where to?	Consumers, services, countries, who talks to whom, destinations, IP lookup, restrictions
Network tree	How is the network wired?	Discovered tree, editable with the mouse (move, re-parent, merge, hide)
Subscribers	Who are the non-PPPoE subscribers?	Static-IP subscribers, subscribers per VLAN
Plans	Which rate for whom?	Default plan, each subscriber's plan and its source, packages pushed by the API
Insights	What to do commercially?	Subscribers at risk of leaving, ready for a bigger plan, sites with room to grow
Devices	Which equipment?	Router health, adding a router, Ubiquiti antennas, radio health, inventory
API	How to integrate?	API keys, links to the API guide and to this documentation
Settings	How does the tool behave?	Accounts, settings, writing to the routers, caps held or not, command log

Read the Settings page before enabling writes

The *Shaping and writing to the routers* section shows, before enabling, what freeQoS would do on each router: queues created, modified, deleted, and links skipped with their reason.

Two banners can appear at the top of the interface:

- **Other instance detected:** another freeQoS installation creates queues on the same routers. Its objects are not touched; one of them must be stopped.
- **Frozen lines:** anti-oscillation has frozen one or more queues. The banner says which and why.

Empty latency

When a subscriber has no latency, the cell does not stay silent: it says whether the subscriber has never answered, whether the measurement is too old, or whether the probe has not run yet. *Find the cause* runs the diagnostic described in section 5.

9. API

Everything the interface does goes through the API. The full guide, with `curl` examples and the reference generated by the server, is at `<your-freeqos-url>/api-guide`. This section gives its structure and rules.

Three APIs, one server, the same keys

API	Path	Use	Units
Operating	<code>/api/v1</code>	Everything the interface does: routers, plans, limits, measurements, traffic, diagnostics	Mbit/s
Model	<code>/model/v1</code>	Commercial inventory: <code>accounts</code> , <code>packages</code> , <code>sites</code> , <code>access_points</code> , <code>services</code> . Same contract as the Preseem model API: an existing integration works by changing the URL and the key.	kbit/s
Usage	<code>/usage/v1</code>	Bytes consumed per sold line, per hour, day or month	bytes

How the commercial model reaches the network

A billing system (Splynx, UISP CRM, Powercode...) knows its customers and what it sold them, but not the routers. The `/model/v1` API makes the link. It takes the exact shape of the Preseem model API: an existing integration only has to change URL and key.

Collection	What it describes	Fields that matter
<code>accounts</code>	A customer (person or company)	<code>id</code> , <code>name</code>
<code>packages</code>	A plan from the catalogue	<code>down_speed</code> , <code>up_speed</code> in kbit/s
<code>sites</code>	A tower, a location	<code>name</code> : it becomes the PoP name in freeQoS
<code>access_points</code>	A radio sector of a site	<code>site</code> , <code>ip_address</code>
<code>services</code>	A sold line: a customer, a plan, an address	<code>account</code> , <code>package</code> , <code>attachments</code> (networks and CPE MAC), <code>parent_device_id</code> (the sector)

What happens when a service is pushed (`PUT /model/v1/services/{id}`):

1. **The rate** is the service's own if it has one, otherwise its package's. A commercial exception on one line does not require creating a package for it.
2. **The PoP** is found by walking service → access point → site → site name. freeQoS finds the router to configure through that name.
3. **With an address** (`network_prefixes`), the service becomes a static-IP subscriber: it is placed on its router, measured and limited.
4. **With only a MAC**, it is set aside. As soon as that MAC appears in a router's ARP or DHCP table, freeQoS deduces the address and places the service.
5. **An identifier already used by a record entered by hand** is refused (`409`): the API never takes over a human action.

`PUT` creates or replaces: billing can replay its whole inventory every night without risk. The identifier in the URL is authoritative; a body carrying another one is refused (`400`).

For a PPPoE subscriber, the simplest is still `PUT /api/v1/plans/{login}` : the login is enough to find it on its router.

Consumption (`/usage/v1`) comes from NetFlow, not from queue counters: those restart from zero at each reconnection and do not exist for a subscriber without a queue. With no router exporting, this API returns zeros and says so in its `source` field.

Authentication

A key is created in the *API* tab. The secret is shown only once. Two scopes:

- **read**: every `GET` ;
- **read + write**: also `POST` , `PUT` , `PATCH` , `DELETE` .

A key can never manage keys, accounts or passwords: that stays behind a person's login.

Three equivalent forms are accepted everywhere:

```
Authorization: Bearer <key>
X-API-Key: <key>
Authorization: Basic base64(<key>:)      # Preseem form
```

`GET /model/v1` checks a key in one call: it returns the collections, the key prefix and its scopes.

Conventions

- JSON in and out. *Down* = towards the subscriber, *up* = from the subscriber.
- ISO 8601 timestamps in UTC.
- A PPPoE subscriber is identified by its login, a static-IP subscriber by its reference, a model object by the `id` chosen in the URL.
- `PUT` creates or replaces: replaying the whole inventory is harmless.
- A plan or limit change is written to the router **in the same call**.

The enforcement report

Every response that touches a queue contains an `enforcement` object telling what actually happened:

state	Meaning
<code>file-posee</code>	Queue written (or already correct)
<code>file-retiree</code>	Queue removed (plan deleted)
<code>file-a-posee</code>	Would be written: simulation

state	Meaning
ecarte	Not written; <code>reason</code> says why (subscriber offline, shared address, infrastructure address...)
conflit	Another queue already targets this subscriber; to be cleaned by hand
sans-routeur	No router carries this subscriber yet
erreur	The router refused or is unreachable; <code>reason</code> gives the RouterOS message

The calls that matter

Need	Call
Add a router	POST <code>/api/v1/pops/routers</code> then GET <code>/api/v1/pops/provisioning/{name}</code>
List subscribers and their plan	GET <code>/api/v1/plans</code>
Give a plan	PUT <code>/api/v1/plans/{login}</code> with <code>down_mbps</code> , <code>up_mbps</code> or <code>package_id</code>
Remove a plan	DELETE <code>/api/v1/plans/{login}</code>
Force a limit (unpaid bill, fair use)	PUT <code>/api/v1/shaping/policies</code>
Temporary boost	POST <code>/api/v1/shaping/boosts</code>
Declare a static-IP subscriber	POST <code>/api/v1/static-clients</code>
Push the commercial inventory	PUT <code>/model/v1/{collection}/{id}</code>
Read consumption	GET <code>/usage/v1/services</code>
A subscriber's latency, diagnostic	GET <code>/api/v1/rtt/diagnose</code>
Overall state	GET <code>/api/v1/status</code>

A few usage rules:

- A plan for an unknown login returns `404` : the subscriber must first be seen on a router or declared.
- Giving a plan lifts a forced limit on that subscriber: the plan becomes the rule again.
- The report of an API call is authoritative. In simulation, automatic writing is blocked even for a `write` key: `state` is then `file-a-poser`.

Reference

`/openapi.json` describes every route, generated by the installed server: it cannot drift from the code. The `/api-guide` page makes it readable, with search.

10. Security

freeQoS holds router credentials that can write across the whole network. Security follows a simple principle: **what writes is rare, explicit and logged**.

Interface accounts

- **First account:** on the first visit, the interface offers to create one. It has the `edit` role.
- **Two roles:** `read` and `edit`. A `read` account gets a `403` from the **server** on any write, whatever the route. Only an `edit` account manages accounts.
- **Passwords:** 12 characters minimum, too-common words are refused. They are hashed with `scrypt` ($N=2^{14}$, $r=8$, $p=1$, salted, compared in constant time).
- **Anti-guessing:** 5 failures in 5 minutes lock the email **and** the client address for 5 minutes. Each repeat doubles the wait, up to 1 h.
- **Session:** an `HttpOnly`, `SameSite=Strict` cookie, valid 24 h, 30 days at most (`SESSION_TTL_HOURS`, `SESSION_MAX_HOURS`). Writes also check `Origin` and `Sec-Fetch-Site`.
- **Log:** every login (successful, refused, locked), logout and account change is recorded with the address and browser. At login, everyone sees the date and origin of their previous login.

HTTP headers

Header	Effect
<code>Content-Security-Policy</code>	Only scripts served by freeQoS run
<code>X-Frame-Options: DENY</code> , <code>frame-ancestors 'none'</code>	The page cannot be embedded elsewhere
<code>Referrer-Policy: same-origin</code>	Internal addresses do not leak to external links
<code>Strict-Transport-Security</code>	HTTPS only: no going back to plain HTTP

API keys

`read` or `read + write` scope. A key never manages keys, accounts or passwords. It can be disabled and revoked. Every write made with a key is logged as `api:<name> (<prefix>)`.

Router credentials

- Passwords are encrypted in the database with `data/secret.key` and never come out of any response, not even encrypted.
- **Least privilege:** a dedicated RouterOS account (`read,write,api,test`), without `ssh`, `ftp`, `winbox` or `policy`.
- **Separate accounts:** `REQUIRE_SEPARATE_WRITE_ACCOUNT=true` requires a distinct account for writing.
- **Encryption:** `api-ssl` (8729) with `tls_verify=strict` or a fingerprint, rather than the clear API, when freeQoS and the routers do not share an isolated management network.

Writing to the network

- Simulation by default; explicit enabling, after reading the plan.
- `ENFORCEMENT_ENABLED=false` in the environment locks writing: the interface cannot turn it back on.
- Only objects of **this** instance are touched; the others are read, never modified.
- The circuit breaker refuses an abnormally large plan.
- The command log (`/api/v1/shaping/audit`) gives the author, target and result of each write.

Network exposure

Port	Use	Who needs access
8000/TCP (or 443 behind a TLS proxy)	Interface and API	Operators, billing system
2055/UDP	NetFlow	The routers only
8728 or 8729/TCP outgoing	RouterOS API	freeQoS towards the routers

It is recommended to put the interface behind HTTPS and to restrict 2055/UDP to the routers' addresses.

11. Operations

Monitoring freeQoS

Call	Answers	Use
GET /health	Does the process answer? Touches neither the database nor the routers.	Liveness (container restart)
GET /health/ready	Does the database answer, is data arriving? 503 if a cycle keeps failing.	Readiness, monitoring
GET /api/v1/status	State of each cycle, instance identifier, other instances seen, frozen lines	Diagnostics
GET /api/v1/status/runs	History of cycle runs	Diagnostics

Monitor /health/ready , not /health : a collector can fail for an hour while the process still answers.

The cycles

Each cycle can be run immediately with POST /api/v1/jobs/{name}/run .

Cycle	Period	Role
collect_subscribers	10 s	Subscriber sessions and rates
collect_links	10 s	Port rates
collect_backhauls	30 s	Radio capacity (UISP, airOS)
probe_rtt	30 s	Latency probe
expire_boosts	30 s	End of boosts
ip_intel	30 s	Naming addresses
netflow_flush	60 s	Writing aggregated flows
reload_inventory	60 s	Re-reading the router list
reconcile_shaping	2 min	Queues: desired versus actual state
refresh_plans	5 min	Re-reading plans
qoe_closed_loop	5 min	QoE loop per sector
traffic_restrictions	5 min	Address lists of restrictions
netflow_export	10 min	NetFlow export on the routers

Cycle	Period	Role
discover_topology	15 min	Network tree

Live settings

Most operating settings can be changed in *Settings* without a restart (`PUT /api/v1/settings/{name}`). They are stored in the database with their history (`GET /api/v1/settings/history : who, what, when`). `DELETE /api/v1/settings/{name}` goes back to the value from `.env` .

Deliberately out of the interface's reach: database access, secrets, and `ENFORCEMENT_ENABLED=false` when it is set in the environment.

Data and retention

Measurements live in TimescaleDB hypertables:

- split by day (`CHUNK_INTERVAL_HOURS=24`);
- compressed after 7 days (`COMPRESSION_AFTER_DAYS`);
- deleted after 90 days (`RETENTION_DAYS`).

Without TimescaleDB, freeQoS runs on plain PostgreSQL, without compression or automatic retention.

Backup and restore

```
make backup                                # backups/YYYYMMDD-HHMMSS/: base.dump + secret.key
make restore DIR=backups/20261008-0300    # replaces the database, asks for confirmation
```

`make restore` stops the application, wraps `pg_restore` in `timescaledb_pre_restore()` / `post_restore()` , puts `secret.key` back, then restarts. Copy backups off the server.

`make backup` does not copy `data/instance.id` . On a new server, copy it back or set `FREEQOS_INSTANCE_ID` ; otherwise queues already in place will be seen as another instance's and left alone.

Common actions

Situation	Action
Start again from scratch on a router (inconsistent queues, old version)	<i>Reset queues</i> in <i>Settings</i> : removes all queues of this instance, unfreezes lines, then reconciliation reinstalls the desired state
Wrong latencies after a change of method	<code>POST /api/v1/rtt/reset-history</code> (clears 48 h of measurements and the scores derived from them)
A router's password changed	<code>PATCH /api/v1/pops/routers/{id}</code> then <code>POST /api/v1/pops/provisioning/{name}</code>
Vanished boxes in the tree	<code>POST /api/v1/topology/forget-stale</code>

Situation	Action
Stop writing urgently	Switch in <i>Settings</i> , or <code>ENFORCEMENT_ENABLED=false</code> then restart
Stop the latency probe	<code>PUT /api/v1/rtt</code>

Several instances

Two installations must not drive the same routers. If it happens, each only touches its own objects (`instance=<id>` comment), but they fight over subscribers. The *Other instance* banner reports it. For an update or a migration, stop the old one **before** starting the new one.

12. Troubleshooting

All these cases come from the lab or the first deployment. For each: the symptom, the check that settles it, then the action.

700 to 900 ms latency on nearby subscribers

- **Cause:** the ping leaves through the `main` table while subscriber routes are in a VRF. It goes up to the core and back.
- **Check:** *Find the cause* → `routing_table` must name the VRF (`CUST-INET`), not `main` . On the router, `/ping <subscriber> vrf=CUST-INET count=5` must give a few ms.
- **Action:** up to date, freeQoS picks the VRF by itself. If `routing_table` stays `main` , look at `route_candidates` : the route to the subscriber may be a default route towards upstream. After fixing, `POST /api/v1/rtt/reset-history` clears the wrong measurements.

Empty latency ("no reply", "probe silent")

Run *Find the cause* and read the `code` (table in section 5). The two most frequent cases:

- `client_blocks_icmp` : the subscriber's CPE filters ping. This is not a degradation; the subscriber is excluded from QoE.
- `no_test_policy` : the RouterOS account's group lacks the `test` policy.

High latency on a saturated subscriber

- **Symptom:** a single subscriber at 300 or 500 ms, filling its plan.
- **Cause:** real bufferbloat. Without a CAKE queue, the queue forms in a buffer along the path (CPE, radio, test link).
- **Action:** give it a plan slightly **below** the real capacity of its line, so the queue forms in CAKE. Example: 800 kbit/s on a line that holds 1 Mbit/s. If latency stays high, the real capacity is lower than expected: lower the plan further.

A queue is rewritten every 2 minutes

Two writers are fighting over the queue: two freeQoS instances, a script, or another queue targeting the same address.

- **Check:** `GET /api/v1/status` . Read `other_instances` (another installation is writing) and `frozen_lines` (anti-oscillation froze the line).
- **Action:** stop the extra instance, then *Reset queues*. If a queue created by hand targets the same subscriber, remove it by hand: freeQoS will not touch it.

A parent queue appears on a transit link

Recent versions skip routing links, the uplink, the gateway and links towards other managed routers.

- **Check:** the *Settings* page lists skipped links with their `skip_reason` .

- **Action:** a queue created by an old version stays until *Reset queues*. If the link is not recognised as transit, correct its role in the tree (`PATCH /api/v1/topology/nodes/{key}`).

Nothing is written to the routers

In order:

1. Is writing enabled? `GET /api/v1/shaping/enforcement` .
2. Is it locked by the environment? `ENFORCEMENT_ENABLED=false` in `.env` .
3. Does the account have `write` ? `GET /api/v1/shaping/capability` .
4. Did the plan trip the circuit breaker? The log says so.
5. Does the subscriber have a plan? A subscriber that is only detected is observed, not limited.

No NetFlow traffic

1. `GET /api/v1/netflow/status` : are datagrams arriving?
2. `GET /api/v1/netflow/export` : is the target set, and to which address?
3. In a container, is `NETFLOW_COLLECTOR_ADDRESS` set?
4. Is UDP port 2055 open between the routers and freeQoS?

Scores stay old

Scores are computed over a sliding 15-minute window. After a measurement fix, old samples still weigh in.

`POST /api/v1/rtt/reset-history` starts again from clean measurements.

The router refuses the connection

`POST /api/v1/pops/routers/test` gives a hint:

Hint	Check
Port closed	<code>api</code> or <code>api-ssl</code> service enabled, and allowed from the freeQoS address
Credentials refused	Account name and password
<code>api</code> policy missing	Group of the RouterOS account
Certificate refused	<code>api-ssl</code> certificate, or <code>tls_verify=fingerprint</code>

13. Settings reference

All settings are read from the environment or from `.env`. The variable name is the one in the table, in capitals. Those marked ***** can also be changed live in *Settings*.

Application and database

Variable	Default	Role
APP_ENV	lab	Environment label
LOG_LEVEL	INFO	Log verbosity
AUTH_ENABLED	true	Login required; only disable on an isolated lab
SESSION_TTL_HOURS / SESSION_MAX_HOURS	24 / 720	Idle session / maximum duration
DATABASE_URL	postgresql://qos:... @localhost:5432/qos	PostgreSQL / TimescaleDB database
DB_POOL_MIN / DB_POOL_MAX	1 / 8	Connections
DB_AUTO_MIGRATE	true	Migrations at startup
CHUNK_INTERVAL_HOURS	24	Hypertable chunking
COMPRESSION_AFTER_DAYS	7	Compression
RETENTION_DAYS	90	Purge
APP_SECRET_KEY_FILE	data/secret.key	Encryption key (generated if missing)
FREEQOS_INSTANCE_ID	data/instance.id	Instance identifier in comments

Inventory and sources

Variable	Default	Role
ROUTERS_FILE / ROUTERS	empty	Routers declared by file or as JSON
BACKHAUL_PROVIDER	mock	uisp, aios or mock (lab simulator). In production, choose uisp or aios.
UIISP_BASE_URL, UIISP_TOKEN	—	UIISP controller (read-only)
AIROS_USERNAME, AIROS_PASSWORD	—	Direct access to the antennas
PLAN_PROVIDER	clients	clients (API and <i>Plans</i> page) or freeradius_sql

Variable	Default	Role
RADIUS_DSN , RADIUS_RATE_ATTRIBUTE	—, Mikrotik-Rate-Limit	Plans read from FreeRADIUS

Cycles

Variable	Default
SUBSCRIBER_INTERVAL_S / LINK_INTERVAL_S	10
BACKHAUL_INTERVAL_S	30
INVENTORY_REFRESH_INTERVAL_S	60
SHAPING_RECONCILE_INTERVAL_S	120
PLAN_REFRESH_INTERVAL_S	300
TOPOLOGY_REFRESH_INTERVAL_S	900
SCHEDULER_ENABLED	true

Latency

Variable	Default	Role
RTT_ENABLED	true	Probe on (switchable live with PUT /api/v1/rtt)
RTT_INTERVAL_S	30	Cycle
RTT_BATCH_SIZE	20	Subscribers per cycle and per PoP
RTT_COUNT / RTT_PING_INTERVAL_MS	5 / 200	Pings per subscriber
RTT_PROBE_DSCP	46	Probe priority (0 = none)
RTT_MAX_AGE_S	300	Maximum age of a displayed measurement
LATENCY_INTERNET_TARGETS	1.1.1.1 , 8.8.8.8	Targets of the internet segment

Plans and shaping

Variable	Default	Role
ENFORCEMENT_ENABLED	false	Writing to the routers; an explicit false = lock
ENFORCEMENT_MAX_ACTIONS *	500	Circuit breaker per plan

Variable	Default	Role
REQUIRE_SEPARATE_WRITE_ACCOUNT *	false	Distinct write account required
DEFAULT_PLAN_DOWN_MBPS / UP *	100 / 20	Default plan
DEFAULT_PLAN_FOR_DETECTED_CLIENTS *	false	Also limit subscribers that are only detected
SHAPING_SAFETY_FACTOR *	0.90	Share of measured capacity given to the parent queue
SHAPING_FLOOR_MBPS *	5	Minimum rate of a parent queue
SHAPING_PRUNE *	true	Remove queues that became useless
SHAPING_ADOPT_FOREIGN_QUEUES *	false	Never take over other people's queues
SHAPING_QUEUE_FOR_DETECTED_LINKS *	true	Parent queues on discovered links
CAKE_OVERHEAD / CAKE_RTT_MS *	22 / 50	CAKE options
CAKE_DIFFSERV , CAKE_FLOWMODE , CAKE_NAT , CAKE_ACK_FILTER , CAKE_WASH , CAKE_MPU *	RouterOS	Advanced CAKE options
BOOST_CHECK_INTERVAL_S	30	Boost expiry

QoE loop

Variable	Default	Role
QOE_LOOP_INTERVAL_S	300	Cycle
QOE_WINDOW_MINUTES	15	Analysis window
QOE_SCORE_THRESHOLD	55	Score below which a subscriber is degraded
QOE_MIN_DEGRADED_SUBSCRIBERS	2	Degraded subscribers needed to act on a sector
QOE_TRIM_STEP / QOE_TRIM_FLOOR	0.10 / 0.50	Tightening step / floor
QOE_RECOVERY_CYCLES	3	Healthy cycles before relaxing

NetFlow and traffic

Variable	Default	Role
NETFLOW_ENABLED / NETFLOW_PORT	true / 2055	Collector

Variable	Default	Role
NETFLOW_FLUSH_INTERVAL_S	60	Writing aggregates
NETFLOW_EXPORT_AUTO *	true	Export set up by freeQoS
NETFLOW_EXPORT_INTERVAL_S	600	How often the export is checked
NETFLOW_COLLECTOR_ADDRESS	auto	Address announced to the routers (required in a container)
NETFLOW_TRACK_DESTINATIONS / NETFLOW_DESTINATION_LIMIT *	true / 2,000	Destinations kept (7 days)
NETFLOW_TRACK_HOSTS / NETFLOW_HOST_LIMIT	false / 500	Unknown hosts seen
IPFINDER_ENABLED *	true	Naming addresses (rDNS, RDAP, GeoIP, each can be disabled live)
IPFINDER_INTERVAL_S / IPFINDER_BATCH_SIZE *	30 / 40	Naming pace (only the batch size is live)
RESTRICTIONS_INTERVAL_S	300	Restriction updates
RESTRICTION_ADDRESS_LIMIT *	5,000	Maximum addresses per list